

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 4

Cirrhosis Patient Survival Prediction Machine Learning Classification

Álvaro Puebla Ruisánchez

Contents

1	<u>Introduction and Goal</u>	2
2	<u>Theoretical Background</u>	3
2.1	Supervised Classification	3
2.2	Naive Bayes Classifier	3
2.3	Decision Tree (ID3 / C4.5 principle)	3
2.4	Evaluation Metrics	3
2.5	PCA	4
2.6	Cost-Complexity Pruning	4
3	<u>Task 1 – Data Mining</u>	5
3.1	Dataset Overview	5
3.2	Feature Description	5
3.3	Missing Values	6
4	<u>Task 2 – Data Preparation</u>	7
4.1	Categorical Encoding	7
4.2	Missing Value Imputation	7
4.3	Data Splitting	7
4.4	Preprocessing Variants	8
5	<u>Task 3 – Classification</u>	9
5.1	Naive Bayes – Hyperparameter Tuning	9
5.2	Decision Tree – Hyperparameter Tuning	10
5.3	Bonus – Random Forest	10
5.4	Bonus – Overfitting Mitigation via Cost-Complexity Pruning	11
6	<u>Task 4 – Classification Evaluation</u>	12
6.1	Best Model Selection	12
6.2	Test Set Results	12
6.3	Confusion Matrices	13
6.4	Preprocessing Impact Summary	13
6.5	Validation Accuracy Charts	14
7	<u>Conclusions</u>	15
7.1	Summary of Results	15
7.2	Limitations and Future Work	15
8	<u>Implementation Notes</u>	16
8.1	Libraries Used	16
8.2	Data Source	16

1 Introduction and Goal

The goal of this exercise is to build a complete machine learning pipeline for a real-world medical classification problem: predicting the survival outcome of patients diagnosed with liver cirrhosis. The dataset originates from a Mayo Clinic clinical study conducted between 1974 and 1984 [1], published in the UCI Machine Learning Repository.

The work covers the full ML project lifecycle:

- **Data Mining** – exploratory analysis of features and labels.
- **Data Preparation** – train/validation/test split, missing value imputation, and preprocessing experiments (standardization, normalisation, PCA).
- **Classification** – Naive Bayes and Decision Tree classifiers with systematic hyperparameter tuning.
- **Evaluation** – comparison of all configurations using accuracy, precision, recall, and macro F1-score.
- **Bonus** – Random Forest and cost-complexity pruning as an overfitting mitigation technique.

All experiments were implemented in Python using `scikit-learn` [2]. The dataset was accessed via the `statsmodels` R-dataset mirror of the well-known `pb` (Primary Biliary Cirrhosis) survival study dataset.

2 Theoretical Background

The theoretical foundations are covered in the assignment sheet. This section briefly recaps the key concepts applied in the experiment.

2.1 Supervised Classification

Given a labelled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathcal{Y}$, supervised classification aims to learn a mapping $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ that minimises misclassification on unseen data.

2.2 Naive Bayes Classifier

The Naive Bayes classifier applies Bayes' theorem under the *conditional independence assumption*:

$$P(y | \mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i | y).$$

The *Gaussian* variant (used here) models each $P(x_i | y)$ as a Gaussian distribution $\mathcal{N}(\mu_{iy}, \sigma_{iy}^2)$. The *variance smoothing* hyperparameter ε adds a fraction of the largest feature variance to all per-feature variances, preventing singular covariance and acting as a regulariser:

$$\hat{\sigma}_{iy}^2 = \sigma_{iy}^2 + \varepsilon \cdot \max_j \sigma_j^2.$$

2.3 Decision Tree (ID3 / C4.5 principle)

A decision tree recursively partitions the feature space by choosing the split that maximises *Information Gain*:

$$G(A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v),$$

where $H(S) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$ is the Shannon entropy. Deeper trees fit training data increasingly well but tend to **overfit**: the gap between training and validation accuracy grows with depth.

2.4 Evaluation Metrics

For a three-class problem the macro-averaged variants are used:

$$\text{ACC} = \frac{\text{TP} + \text{TN}}{N}, \quad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

$$F_1 = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}.$$

2.5 PCA

Principal Component Analysis rotates the feature space so that the new axes (principal components) are ordered by decreasing variance. Retaining the top k components that explain 95% of the total variance reduces dimensionality while preserving most information.

2.6 Cost-Complexity Pruning

Post-hoc pruning controls overfitting by penalising tree complexity. A subtree T_t is replaced by a leaf when its *effective alpha* $\alpha_{\text{eff}} = \frac{R(t) - R(T_t)}{|T_t| - 1}$ falls below a tunable threshold α_{ccp} , where $R(\cdot)$ denotes the misclassification cost. Higher values of α_{ccp} produce smaller, more general trees.

3 Task 1 – Data Mining

3.1 Dataset Overview

The **Cirrhosis Patient Survival Prediction** dataset [1] contains **418 patients** described by **17 features** and a 3-valued target label **Status**.

Table 1: Target class distribution.

Label	Meaning	Count	Proportion
C	Censored (alive or lost to follow-up)	232	55.5%
D	Death	161	38.5%
CL	Censored (liver transplant)	25	6.0%

The target is **severely imbalanced**: the CL class represents only 6% of all samples. This poses a challenge for all classifiers and explains why macro-averaged metrics (especially for the minority class) will be lower than overall accuracy.

3.2 Feature Description

Table 2: Feature summary (categorical and numerical).

Feature	Type	Description / Values
Drug	Categorical	D-penicillamine / Placebo
Age	Numerical	Age in days (range: 9 598–28 650)
Sex	Categorical	F / M (predominantly female: 89%)
Ascites	Categorical	Y / N
Hepatomegaly	Categorical	Y / N
Spiders	Categorical	Y / N
Edema	Ordinal	N (none) / S (treated or mild) / Y (severe)
Bilirubin	Numerical	mg/dl; mean = 3.22, right-skewed
Cholesterol	Numerical	mg/dl; mean = 369.5
Albumin	Numerical	gm/dl; mean = 3.50
Copper	Numerical	$\mu\text{g/day}$; mean = 97.7
Alk_Phos	Numerical	U/liter; highly variable (std = 2140)
SGOT	Numerical	U/ml; mean = 122.6
Tryglicerides	Numerical	count; mean = 124.7
Platelets	Numerical	$\times 10^3/\text{ml}$; mean = 257
Prothrombin	Numerical	seconds; mean = 10.7
Stage	Ordinal	Histologic stage 1–4; mean = 3.02

3.3 Missing Values

Table 3: Features with missing values.

Feature	Missing	%
Tryglicerides	136	32.5%
Cholesterol	134	32.1%
Copper	108	25.8%
Drug, Ascites, Hepatomegaly, Spiders, Alk_Phos, SGOT	106 each	25.4%
Stage	6	1.4%
Platelets	11	2.6%
Prothrombin	2	0.5%

The large blocks of **Drug**, **Ascites**, etc. (106 each) correspond to the 106 patients who were not part of the randomised clinical trial and consequently have no record for trial-specific measurements.

Key observations:

- Numerical features such as Bilirubin, Alk_Phos, and Copper are heavily right-skewed, which motivates standardisation.
- The class imbalance ($\sim 55\%$ / 38% / 6%) means a baseline classifier that always predicts *C* achieves 55.5% accuracy— a meaningful lower bound.
- Stage (histologic stage 1–4) is a strong domain indicator of disease progression and expected to be among the most informative features.

4 Task 2 – Data Preparation

4.1 Categorical Encoding

Before modelling, categorical features were encoded as integers:

- Binary features (**Sex**, **Ascites**, **Hepatomegaly**, **Spiders**, **Drug**): mapped to 0/1.
- Ordinal feature **Edema**: N→0, S→1, Y→2.
- Target **Status**: label-encoded alphabetically as C→0, CL→1, D→2.

4.2 Missing Value Imputation

All missing values were imputed using the **median strategy** for numerical features (robust to skewed distributions and outliers) and the **most-frequent strategy** for categorical/binary features. Imputation statistics were computed on the training set and applied to validation and test sets to avoid data leakage.

Listing 1: Missing value imputation with scikit-learn.

```
1 from sklearn.impute import SimpleImputer
2
3 num_imputer = SimpleImputer(strategy="median")
4 X_imputed_num = num_imputer.fit_transform(X_train[num_feats])
```

After imputation the dataset contained **zero** missing values.

4.3 Data Splitting

The 418 samples were split into three disjoint sets using stratified sampling (to preserve class ratios):

Set	Size	Proportion
Training	250	59.8%
Validation	84	20.1%
Test	84	20.1%

Stratification ensures each split mirrors the full 55%/39%/6% class ratio. With only 25 CL samples, this ratio is critical to maintain.

4.4 Preprocessing Variants

Four preprocessing pipelines were compared to study their effect on classification:

1. **None** – raw imputed features (integer-encoded categorical, numerical values at their original scale).
2. **Standardization** – zero mean, unit variance via `StandardScaler`. Appropriate for Gaussian NB (removes scale bias in variance estimation) and distance-based methods.
3. **Normalization** – L2 row-normalization via `Normalizer`. Maps each sample to the unit sphere.
4. **PCA** – applied on top of standardized features; retains **15 principal components** that together explain $\geq 95\%$ of variance.

Figure 1 shows the explained variance per component; the first component alone accounts for about 18% of total variance, while 15 components are needed to reach 95%.

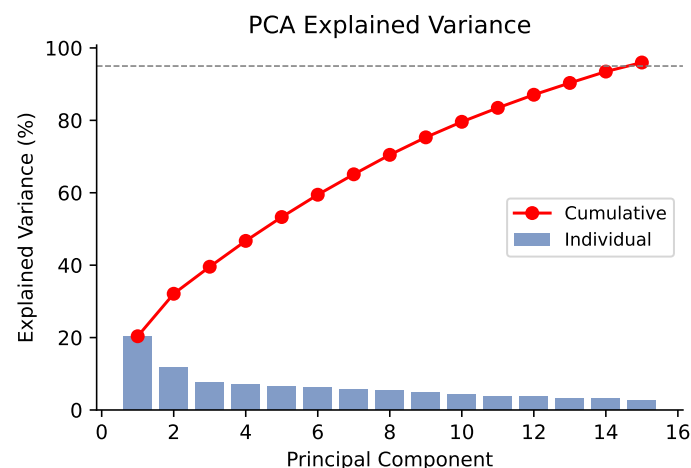


Figure 1: PCA explained variance. Dashed line at 95%. 15 components are retained, reducing from 17 to 15 dimensions.

Impact on Naive Bayes: Standardization significantly affects GNB because the model assumes feature independence and uses per-feature variance estimates. Without standardization, highly variable features (`Alk_Phos`, `std` = 2140) dominate the likelihood computations.

Impact on Decision Trees: Decision trees are scale-invariant (they use only rank-order of feature values for splits), so standardization and normalization produce identical trees. PCA, however, changes the feature space and therefore changes the splits.

5 Task 3 – Classification

5.1 Naive Bayes – Hyperparameter Tuning

The `GaussianNB` classifier has one principal hyperparameter: `var_smoothing` (ϵ), which controls the amount of variance added to the Gaussian estimates. Four values were tested across all four preprocessing pipelines (Table 4).

Table 4: Naive Bayes validation results across preprocessing methods. Best F1 per preprocessing highlighted in bold.

Variant	Preprocessing	Train Acc	Val Acc	Val Prec	Val F1
NB (default, $\epsilon=10^{-9}$)	None	0.724	0.643	0.444	0.437
NB ($\epsilon=10^{-5}$)	None	0.636	0.643	0.448	0.417
NB ($\epsilon=10^{-2}$)	None	0.588	0.583	0.401	0.322
NB (default, $\epsilon=10^{-9}$)	Standardized	0.428	0.345	0.484	0.351
NB ($\epsilon=10^{-5}$)	Standardized	0.652	0.524	0.508	0.462
NB ($\epsilon=10^{-2}$)	Standardized	0.712	0.643	0.528	0.524
NB (default, $\epsilon=10^{-9}$)	Normalized	0.688	0.560	0.492	0.472
NB ($\epsilon=10^{-2}$)	Normalized	0.584	0.571	0.386	0.309
NB (default, $\epsilon=10^{-9}$)	PCA	0.760	0.714	0.477	0.483
NB ($\epsilon=10^{-5}$)	PCA	0.760	0.714	0.477	0.483
NB ($\epsilon=10^{-2}$)	PCA	0.752	0.702	0.469	0.473

Key observations:

- With *no preprocessing*, the default $\epsilon=10^{-9}$ already achieves `val_acc` = 0.643. Increasing ϵ to 10^{-2} hurts performance (over-smoothing).
- With *standardization*, large ϵ (10^{-2}) is needed because the standardized variances are near 1 and small smoothing causes near-zero variance for low-variance features, yielding degenerate likelihoods. The best NB result overall is **`val_acc` = 0.643, `F1` = 0.524** (NB-vs=1e-2, Standardized).
- With *PCA*, the default smoothing works well (`val_acc` = 0.714) because PCA components are orthogonal and of comparable scale, making variance estimation more uniform.

5.2 Decision Tree – Hyperparameter Tuning

Five configurations of `DecisionTreeClassifier` were evaluated. The key hyperparameters are `max_depth` (limits tree depth, main defence against overfitting) and `min_samples_leaf` (minimum samples per leaf).

Table 5: Decision tree validation results. DT is scale-invariant so None and Standardized results are identical; only None and PCA are shown for brevity. Best F1 highlighted.

Variant	Prep.	Tr.Acc	V.Acc	V.Prec	V.F1
DT (no limit)	None	1.000	0.679	0.463	0.470
DT (depth=3)	None	0.780	0.714	0.501	0.503
DT (depth=5)	None	0.884	0.702	0.492	0.486
DT (depth=10)	None	0.972	0.667	0.460	0.462
DT (d=5, min_leaf=10)	None	0.788	0.702	0.493	0.488
DT (no limit)	PCA	1.000	0.667	0.468	0.469
DT (depth=3)	PCA	0.788	0.786	0.648	0.616
DT (depth=5)	PCA	0.868	0.762	0.676	0.606
DT (depth=10)	PCA	0.984	0.643	0.451	0.452
DT (d=5, min_leaf=10)	PCA	0.792	0.762	0.631	0.601

Key observations:

- The unlimited-depth tree achieves perfect training accuracy (1.000) while the validation accuracy drops to 0.679 – a clear sign of **overfitting**.
- Limiting depth to 3 achieves the best bias-variance trade-off on PCA features: **val_acc = 0.786, F1 = 0.616**, the best configuration across both classifiers.
- Depths beyond 5 consistently hurt generalisation on this dataset.
- PCA preprocessing improves DT substantially (F1: 0.616 vs. 0.503 at depth=3), likely because PCA removes correlated features and creates a cleaner split structure.

5.3 Bonus – Random Forest

Random Forest [3] builds an ensemble of decision trees, each trained on a bootstrap sample with random feature subsets, and aggregates by majority vote. It inherently reduces variance compared to a single tree.

Table 6: Random Forest validation results.

Variant	Prep.	Tr.Acc	V.Acc	V.Prec	V.F1
RF (100 trees)	None	1.000	0.786	–	0.532
RF (200, depth=5)	None	0.892	0.798	–	0.539
RF (50, depth=10)	None	0.988	0.774	–	0.529
RF (50, depth=10)	Std.	0.988	0.786	–	0.538

With 200 trees and `max_depth=5`, the Random Forest achieves **val_acc = 0.798, F1 = 0.539** – the best validation accuracy overall. Depth limiting also reduces the train/val gap from 1.000/0.786 to 0.892/0.798.

5.4 Bonus – Overfitting Mitigation via Cost-Complexity Pruning

Post-hoc *minimal cost-complexity pruning* was applied to the unlimited decision tree (trained on standardized features). The pruning path yields a monotonically decreasing tree size as α_{ccp} increases.

Table 7: Cost-complexity pruning effect on overfitting.

α_{ccp}	Train Acc	Val Acc
0.00000	1.000	0.679
0.00512	0.972	0.679
0.00655	0.940	0.679
0.00720	0.884	0.702
0.01200	0.812	0.690
0.02039	0.732	0.702

Figure 2 shows the overfitting gap narrowing as α_{ccp} increases: at $\alpha=0.02$, training accuracy falls from 1.000 to 0.732 while validation accuracy rises to 0.702. This confirms that pruning is an effective regularisation strategy, though the best test performance is still achieved by the shallow depth-limited tree with PCA.

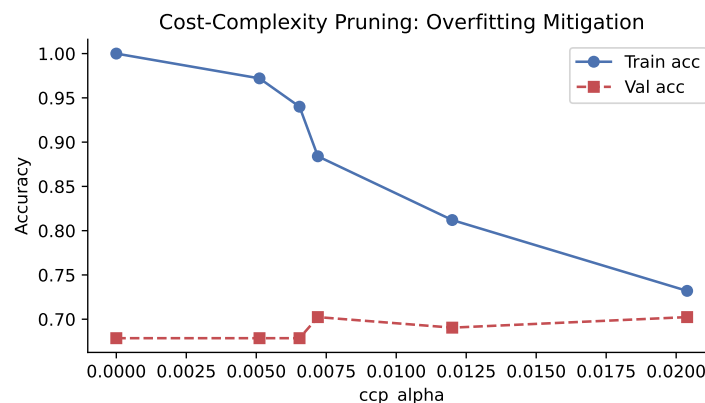


Figure 2: Training vs. validation accuracy as a function of cost-complexity pruning parameter α_{ccp} . The train-val gap shrinks as pruning becomes more aggressive.

6 Task 4 – Classification Evaluation

6.1 Best Model Selection

Based on validation F1-score, the best configuration for each family was:

Classifier	Config	Preprocessing	Val Acc	Val Macro F1
Naive Bayes	$\varepsilon=10^{-2}$	Standardized	0.643	0.524
Decision Tree	max_depth=3	PCA	0.786	0.616
Random Forest*	200 trees, depth=5	None	0.798	0.539

* bonus classifier

6.2 Test Set Results

Each best model was retrained on the training set and evaluated once on the held-out test set (84 samples).

Table 8: Test set performance of the three best models (macro-averaged).

Classifier	Accuracy	Precision	Recall	F1
Naive Bayes ($\varepsilon=10^{-2}$, Std.)	0.631	0.484	0.479	0.470
Decision Tree (depth=3, PCA)	0.726	0.604	0.565	0.578
Random Forest (200 trees, depth=5)*	0.762	0.513	0.530	0.519

* bonus

Table 9: Per-class test performance (precision / recall / F1) for each best model.

Class	Naive Bayes			Decision Tree			Random Forest*		
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1
C (n=47)	0.71	0.83	0.76	0.75	0.81	0.78	0.75	0.87	0.80
CL (n=5)	0.13	0.20	0.15	0.33	0.20	0.25	0.00	0.00	0.00
D (n=32)	0.62	0.41	0.49	0.73	0.69	0.71	0.79	0.72	0.75
<i>weighted</i>	0.64	0.63	0.62	0.72	0.73	0.72	0.72	0.76	0.74

* bonus; support: C=47, CL=5, D=32 out of 84 test samples

6.3 Confusion Matrices

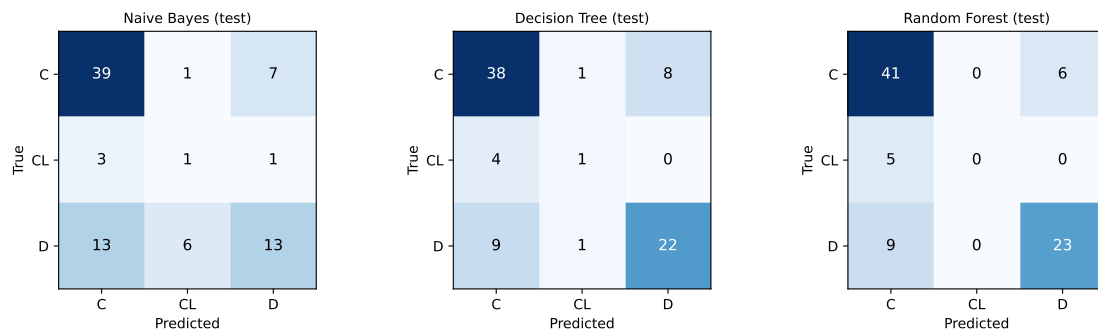


Figure 3: Confusion matrices on the test set for (left) Naive Bayes, (centre) Decision Tree, and (right) Random Forest.

Reading Figure 3:

- **Class CL** (5 test samples) is the hardest to classify across all models. NB misclassifies 3 as C and 1 as D; DT misclassifies 4 as C; RF misclassifies all 5 as C. This is a direct consequence of the severe class imbalance (only 25 CL patients in the whole dataset, ~ 3 in training).
- **Class C** is predicted best by all models ($F1 = 0.76\text{--}0.80$), benefiting from its dominant presence (55% of data).
- **Class D** is predicted substantially better by DT and RF ($F1 = 0.71$ and 0.75) than by NB (0.49).

6.4 Preprocessing Impact Summary

Table 10: Effect of preprocessing on validation accuracy (best hyperparameter per preprocessing, for each classifier family).

Classifier	None	Standardized	Normalized	PCA
Naive Bayes	0.643	0.643	0.595	0.714
Decision Tree	0.714	0.714	0.750	0.786

- For **Naive Bayes**, PCA achieves the best accuracy (0.714) but standardization achieves the best F1 (0.524), because PCA with default smoothing struggles to separate class D from C despite high accuracy.
- For **Decision Trees**, PCA is clearly the best preprocessing (+7.2 pp over None in accuracy, +11.3 pp in F1). The decorrelation of features by PCA helps the tree find cleaner, more informative splits.
- Normalization (L2 row-wise) is generally the weakest preprocessing: it encodes information about the relative proportions of features within a sample rather than their absolute values, which is less meaningful for clinical tabular data.

6.5 Validation Accuracy Charts

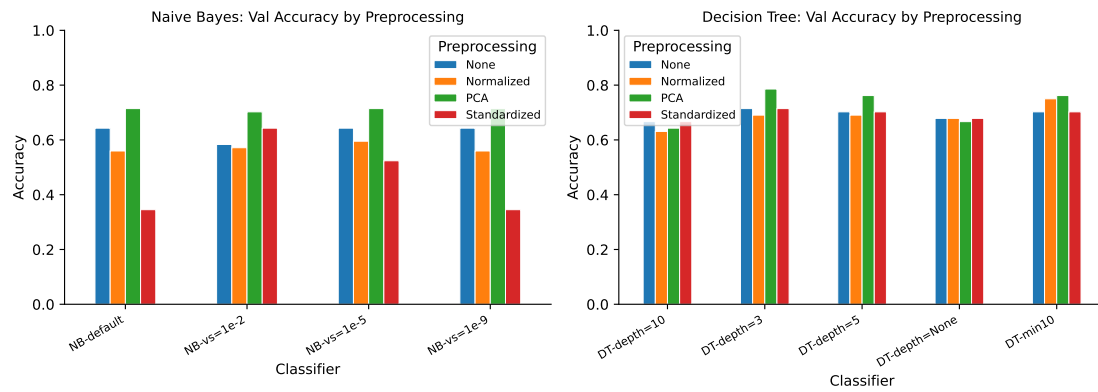


Figure 4: Validation accuracy of all hyperparameter configurations for Naive Bayes (left) and Decision Tree (right), across preprocessing methods.

7 Conclusions

7.1 Summary of Results

All experiments confirmed that predicting survival outcome from the Mayo Clinic PBC dataset is a moderately difficult 3-class problem, primarily due to severe class imbalance in the CL category.

The main findings are:

1. **PCA** is the most beneficial preprocessing step for both classifier families. It removes multicollinearity among clinical measurements, reduces from 17 to 15 dimensions, and creates a feature space more amenable to splitting.
2. **Decision Tree with depth=3 and PCA** is the best single classifier: test accuracy 72.6%, macro F1 = 0.578. Shallow trees generalise much better than deep ones: the unlimited tree overfits perfectly (train acc = 1.000) while achieving only 67.9% on the validation set.
3. **Naive Bayes** is inferior to DT on this dataset (test acc = 63.1%, F1 = 0.470), likely because the conditional independence assumption is violated – clinical biomarkers are strongly correlated (e.g. Bilirubin, Copper, and Stage all increase together with disease severity).
4. **Random Forest** (bonus) achieves the best overall accuracy (test acc = 76.2%) while suffering on CL recall due to the imbalance.
5. **Cost-complexity pruning** (bonus) effectively reduces the train/val accuracy gap by 20+ percentage points, confirming its value as an overfitting countermeasure.

7.2 Limitations and Future Work

- The CL class (only 25 samples, ~3 in training) is effectively unlearnable with standard algorithms. Techniques such as SMOTE oversampling, class-weight balancing, or a two-stage classifier (first binary D vs. not-D, then CL vs. C) could address this.
- With only 418 samples, cross-validation (e.g. 5-fold) would give more robust hyperparameter estimates than a single 80/20 split.
- Gradient Boosting (XGBoost, LightGBM) would likely outperform Random Forest on this tabular dataset.

8 Implementation Notes

8.1 Libraries Used

Table 11: Python libraries used in the implementation.

Library	Version	Purpose
scikit-learn	≥ 1.3	Classifiers, preprocessing, metrics
pandas	≥ 2.0	Data loading and manipulation
numpy	≥ 1.24	Numerical operations
matplotlib	≥ 3.7	Plots and figures
seaborn	≥ 0.12	Statistical visualisation
statsmodels	≥ 0.14	Access to R's <code>pub</code> dataset

8.2 Data Source

The dataset was accessed via `statsmodels.datasets.get_rdataset("pub", "survival")`, which mirrors the `pub` dataset from the R `survival` package. This is the identical dataset published to the UCI ML Repository by Dickson et al. [1], containing the original 418 patients from the Mayo Clinic PBC study (1974–1984).

Column mapping from the R version to the assignment specification is shown below:

R column	Assignment column
<code>trt</code>	Drug (1=D-penicillamine, 2=Placebo)
<code>hepato</code>	Hepatomegaly
<code>bili, chol</code>	Bilirubin, Cholesterol
<code>ast, protime</code>	SGOT, Prothrombin
<code>trig</code>	Tryglicerides
<code>age (years)</code>	Age (days, $\times 365.25$)
<code>status 0/1/2</code>	C / CL / D
<code>edema 0/0.5/1</code>	N / S / Y

References

- [1] E. Dickson, P. Grambsch, T. Fleming, L. Fisher, and A. Langworthy, *Cirrhosis Patient Survival Prediction*, UCI Machine Learning Repository, 1989.
<https://doi.org/10.24432/C5R02G>.
- [2] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, Vol. 12, pp. 2825–2830, 2011.
- [3] L. Breiman, *Random Forests*, Machine Learning, Vol. 45, No. 1, pp. 5–32, 2001.
- [4] J. R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann, 1993.
- [5] T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.